

# Persistence Beats Complexity: A Unified Benchmarking Framework for Short-Term Load Forecasting Across Statistical, Machine Learning, Deep Learning, and Transformer Architectures

Arhaansh Jhingan

*School of Computer Science and Engineering  
Vellore Institute of Technology  
Vellore, India*

arhaansh.ritesh2025@vitstudent.ac.in

M V Chilukuri

*School of Electrical Engineering  
Vellore Institute of Technology  
Vellore, India*

mahendra.chilukuri@vit.ac.in

Ganesan K

*School of Computer Science Engineering  
and Information Systems  
Vellore Institute of Technology  
Vellore, India*

kganesan@vit.ac.in

**Abstract**—Short-term load forecasting (STLF) is fundamental to reliable power system operation, yet rigorous cross-family benchmarking under identical experimental conditions remains scarce. This paper presents a unified framework that evaluates seventeen models spanning six architectural families—persistence baselines, statistical methods, classical machine learning, deep learning, transformers, and a hybrid residual model under an identical preprocessing pipeline, feature engineering strategy, and evaluation protocol. Using seven years of 15-minute resolution electricity demand data from the Haryana state grid (India), all models consume 45 engineered features constructed through multi-scale lag generation, rolling statistics, cyclical encoding, and Fourier harmonics. The benchmark reveals that simple persistence forecasting achieves a MAPE of 1.63%, consistently outperforming classical ML (1.79–2.16%), deep learning (2.00–23.32%), transformer (2.45–3.58%), and hybrid (1.79%) architectures. This counterintuitive finding motivates future investigation into the diminishing returns of architectural complexity in high-resolution load forecasting.

**Index Terms**—Short-term load forecasting, benchmarking, gradient boosting, deep learning, time series forecasting

## I. INTRODUCTION

Short-term load forecasting (STLF) is critical for real-time grid balancing, reserve scheduling, and frequency regulation [1], [2]. The methodological landscape spans classical statistical models such as ARIMA and SARIMA [3], feature-engineered machine learning including gradient boosting and ensemble methods [4], deep learning architectures such as LSTM, GRU, and TCN [5], [6], and hybrid frameworks combining ML baselines with recurrent error correctors [7]. However, most published studies evaluate models under heterogeneous preprocessing conditions, rendering cross-study comparison unreliable [8]. This paper addresses this gap through a unified benchmarking study that evaluates seventeen models across six architectural families—persistence baselines, statistical models, classical ML, deep learning, transformers, and a hybrid residual model under an identical end-to-end pipeline encompassing 45 engineered features, chronological partitioning, and evaluation on 35,040 test observations in original

MW units using MAE, RMSE, and MAPE. The benchmark reveals that simple persistence forecasting achieves the lowest MAPE (1.63%) on 15-minute resolution data, consistently outperforming all complex architectures and motivating further investigation into the diminishing returns of model complexity.

## II. LITERATURE REVIEW

STLF has been addressed through progressively complex architectures. ARIMA and SARIMA remain widely adopted for univariate forecasting due to their interpretability, though they degrade over longer horizons owing to linearity constraints [3]. Gradient boosting frameworks such as XGBoost, LightGBM, and CatBoost have demonstrated strong performance on tabular formulations [4], with competition results confirming that regression models can outperform transformers [9]. Recurrent architectures including LSTM and GRU have been applied for sequential dependency modeling [5], while transformer-based approaches such as PatchTST [10] have attracted interest despite inconsistent domain-specific performance. Hybrid architectures combining ML baselines with deep learning correctors have also been explored [7], though their superiority over well-tuned standalone methods remains empirically mixed. Critically, controlled benchmarking studies remain sparse, with most comparisons evaluating few model families under differing preprocessing conditions [8].

## III. RESEARCH METHODOLOGY

### A. Overall Forecasting Framework

Raw 15-minute resolution electricity demand data from the Haryana state grid undergoes sequential stages of data cleaning, feature engineering, scaling, and sliding window construction before being consumed by seventeen forecasting models across six architectural families. All models produce predictions that are inverse-scaled to original Megawatt (MW) units prior to evaluation using identical metrics and an identical chronological test partition.

## B. Data Preprocessing

The raw dataset comprises approximately 245,000 observations spanning seven years of Haryana state electricity demand, sampled at 15-minute intervals ( $f = 15$  min,  $N_d = 96$  intervals per day). Each daily observation is extracted from structured Excel reports using dynamic header discovery and causal timestamp construction.

**Chronological Ordering and Deduplication.** Daily files are concatenated and sorted by timestamp. Duplicate timestamps are removed, retaining the first occurrence. A strict 15-minute frequency is enforced using `asfreq()`, which inserts NaN markers at missing intervals.

**Missing Value Handling.** Short gaps ( $\leq 4$  intervals, i.e., 1 hour) are imputed via forward-fill to maintain causal integrity. The remaining gaps are linearly interpolated with backward-filling as a final safeguard.

**Outlier Treatment.** Winsorization is applied at  $4\sigma$  from the global mean. Values exceeding  $\mu \pm 4\sigma$  are clipped to the boundary, preserving extreme but plausible demand spikes while eliminating sensor artifacts.

**Feature Scaling.** Two independent `StandardScaler` instances are fitted exclusively on the training partition: one for the 45-dimensional feature matrix  $\mathbf{X}$  and one for the target variable  $y$ . This prevents information leakage from the test set.

**Sliding Window Construction.** The feature-engineered time series is transformed into a supervised learning format using a sliding window of length  $W = 192$  (48 hours). For each index  $i$ , the input-output pair is defined as:

$$\mathbf{X}_i = \{f_i, f_{i+1}, \dots, f_{i+W-1}\}, \quad y_i = \text{load}_{i+W}$$

**Train/Test Split.** The final 35,040 observations (exactly one calendar year) are reserved as the test partition. The remaining data constitutes the training set. Strict chronological ordering is verified to ensure no temporal overlap.

## C. Feature Engineering

The pipeline generates 45 engineered features across seven categories, each designed to capture distinct aspects of load dynamics.

**Calendar Features.** Hour of day, day of week, and month of year provide categorical temporal context. A binary week-end indicator encodes structural demand differences between weekdays and weekends.

**Cyclical Encoding.** Sine and cosine transformations preserve periodic continuity:

$$f_{\sin} = \sin\left(\frac{2\pi \cdot x}{X_{\max}}\right), \quad f_{\cos} = \cos\left(\frac{2\pi \cdot x}{X_{\max}}\right)$$

Applied to hour ( $X_{\max} = 24$ ) and day of week ( $X_{\max} = 7$ ), these encodings ensure that adjacent time periods (e.g., hours 23 and 0) maintain proximity in feature space.

**Lag Features.** Multi-scale lags at offsets  $\{1, 2, 3, 4, 8, 12, 24, 48, 96, 192, 672\}$  capture short-term autocorrelation (lags 1–4), intra-day patterns (lags 24–96), and weekly periodicity (lag 672).

**Rolling Statistics.** Mean, standard deviation, minimum, and maximum are computed over windows of 4 (1 hour), 96 (1 day), and 672 (1 week) intervals. All rolling computations use shifted values ( $L_{t-1}$ ) to prevent data leakage.

**Difference Features.** Load differences  $\Delta L_{t,k} = L_{t-1} - L_{t-k-1}$  for  $k \in \{1, 96, 672\}$  capture short-term momentum, daily, and weekly demand shifts.

**Ratio Features.** The day-over-day ratio  $R_{t,96} = L_{t-1}/(L_{t-96} + \epsilon)$  with  $\epsilon = 10^{-6}$  quantifies relative demand changes.

**Fourier Harmonics.** Three harmonics for daily ( $P = 96$ ) and weekly ( $P = 672$ ) cycles provide smooth periodic representations:

$$F_{k,\sin} = \sin\left(\frac{2\pi kt}{P}\right), \quad F_{k,\cos} = \cos\left(\frac{2\pi kt}{P}\right), \quad k \in \{1, 2, 3\}$$

**Feature Selection.** A union of the top-30 features from mutual information regression and Random Forest importance, augmented with mandatory seasonal features (lag\_96, lag\_672, hour\_sin, hour\_cos), yields the final feature set.

## D. Preprocessing Algorithm

### Algorithm 1: Preprocessing Algorithm

- 1: **Input:** Raw load time series  $\mathcal{L} = \{(t_i, y_i)\}_{i=1}^{N_{\text{raw}}}$  in MW, Timestamp vector  $\mathbf{T}$ , Window length  $W = 192$ , Forecast horizon  $H = 1$ , Rolling windows  $\mathcal{W} = \{4, 96, 672\}$ , Lags  $\Lambda = \{1, 2, 3, 4, 8, 12, 24, 48, 96, 192, 672\}$ , Fourier harmonics  $K = 3$ , periods  $\mathcal{P} = \{96, 672\}$
- 2: **Output:** Training tensors  $\mathbf{X}_{\text{train}} \in \mathbb{R}^{N_{\text{tr}} \times W \times d}$ ,  $\mathbf{y}_{\text{train}} \in \mathbb{R}^{N_{\text{tr}}}$ , Test tensors  $\mathbf{X}_{\text{test}} \in \mathbb{R}^{N_{\text{te}} \times W \times d}$ ,  $\mathbf{y}_{\text{test}} \in \mathbb{R}^{N_{\text{te}}}$
- 3: Sort  $\mathcal{L}$  chronologically by  $\mathbf{T}$  and remove duplicates
- 4: Enforce 15-minute frequency; insert NaT at missing intervals
- 5: Forward-fill missing values (limit  $\leq 4$  intervals)
- 6: Compute  $\mu, \sigma$  of  $y$  and clip  $y_i \leftarrow \text{clip}(y_i, \mu - 4\sigma, \mu + 4\sigma)$  {▷ Winsorization}
- 7: Interpolate remaining NaNs linearly; backward-fill edges
- 8: **Feature Engineering:**
- 9: Extract calendar: hour, day\_of\_week, month, is\_weekend
- 10: Compute cyclical features (sin/cos) for hour and day\_of\_week
- 11: **for** each  $\ell \in \Lambda$  **do**
- 12:   Generate  $\text{lag}_{\ell}(t) \leftarrow y_{t-\ell}$
- 13: **end for**
- 14: **for** each  $w \in \mathcal{W}$  **do**
- 15:   Compute rolling stats (mean, std, min, max) on  $y_{t-1}$  {▷ Shifted}
- 16: **end for**
- 17: Compute differences  $\Delta L_{t,k}$  for  $k \in \{1, 96, 672\}$  on  $y_{t-1}$
- 18: Compute ratio  $R_{t,96} \leftarrow y_{t-1}/(y_{t-96} + 10^{-6})$
- 19: **for** each  $k \in \{1, \dots, K\}$ ,  $P \in \mathcal{P}$  **do**

```

20:   Generate   Fourier   harmonics    $\sin(2\pi kt/P)$ ,
       $\cos(2\pi kt/P)$ 
21: end for
22: Drop rows containing NaN due to lag/rolling initialization
23: Feature Selection:
24: Select top-30 features via  $MI \cup RF$  importance
25: Add mandatory seasonal features to yield final  $d$  features
   (e.g.,  $d = 45$ )
26: Scaling and Splitting:
27: Partition:  $\mathcal{D}_{\text{train}} \leftarrow \mathcal{L}[1 : N - 35040]$ ,  $\mathcal{D}_{\text{test}} \leftarrow \mathcal{L}[N - 35039 : N]$ 
28: Fit  $\text{Scaler}_X$ ,  $\text{Scaler}_y$  exclusively on  $\mathcal{D}_{\text{train}}$  and apply scaling
   to both partitions
29: Sliding Window Construction:
30: for each partition  $\mathcal{D} \in \{\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{test}}\}$  do
31:   for  $i = 1$  to  $|\mathcal{D}| - W$  do
32:      $\mathbf{X}_i \leftarrow [\mathbf{f}_i, \mathbf{f}_{i+1}, \dots, \mathbf{f}_{i+W-1}]$     $\{\triangleright \mathbf{X}_i \in \mathbb{R}^{W \times d}\}$ 
33:      $y_i \leftarrow \text{load}_{i+W}$ 
34:   end for
35: end for
36: return  $(\mathbf{X}_{\text{train}}, \mathbf{y}_{\text{train}}, \mathbf{X}_{\text{test}}, \mathbf{y}_{\text{test}})$ 

```

---

#### E. Benchmark Models

**Persistence.** The simplest baseline predicts  $\hat{y}_t = y_{t-1}$ , exploiting short-term autocorrelation in 15-minute data with zero training.

**Statistical Models.** Seasonal Naïve tiles the last observed daily cycle. SARIMA(2, 1, 2)(1, 1, 1)<sub>96</sub> and SARIMAX (with cyclical exogenous features) are fitted on 10,000 recent observations.

**Classical Machine Learning.** Seven models are evaluated on the tabular sliding window dataset: Linear Regression, Ridge ( $\alpha$  grid search), Lasso, Random Forest, Gradient Boosting, XGBoost, LightGBM, and CatBoost. Linear/Ridge models consume the flattened 192-step tensor; polynomial variants use only the latest timestep.

**Deep Learning.** Four architectures process 3D tensors ( $N \times 192 \times 45$ ): two-layer LSTM (64→32), GRU (64→32), CNN-LSTM (32 filters, kernel 3, LSTM 64), and TCN (4 dilated causal blocks, 32 channels), all trained with Adam ( $\eta = 0.001$ ), MSE loss, gradient clipping, and early stopping (patience 10).

**Transformer Models.** A Baseline Transformer Encoder ( $d_{\text{model}} = 64$ , 4 heads, 4 layers) and PatchTST (patch length 16, stride 8, channel-independent) are trained on a 30,000-sample subset with Adam ( $\eta = 0.0005$ ) and early stopping (patience 5).

**Hybrid Model.** LightGBM+GRU: a two-layer GRU (64→32 with LayerNorm) learns to predict LightGBM residuals. The final forecast is  $\hat{y}_{\text{final}} = \hat{y}_{\text{LGBM}} + \hat{r}_t$ .

#### F. Experimental Protocol

**Chronological Integrity.** All splits strictly respect temporal ordering. The test partition (last 35,040 observations) is never exposed during training or validation.

**Identical Preprocessing.** Every model family consumes features derived from the same preprocessing pipeline. Deep learning and transformer models receive 3D sequence tensors; classical ML models receive the flattened or tabular equivalent.

**Uniform Evaluation.** All predictions are inverse-scaled to MW units. MAE, RMSE, and MAPE are computed on the identical test partition, enabling direct cross-family comparison.

**Hyperparameter Tuning.** Ridge regularization ( $\alpha \in \{0.1, 1.0, 10.0, 100.0\}$ ) and polynomial degree are selected via validation RMSE. Deep learning and transformer models use learning rate schedulers and early stopping as implicit regularization.

**Reproducibility.** A fixed random seed (42) governs all stochastic operations. Deterministic CUDA settings are enforced for PyTorch models.

### IV. RESULTS AND ANALYSIS

#### A. Evaluation Metrics

Three complementary metrics are employed to assess forecasting performance. All metrics are computed on inverse-scaled predictions in original Megawatt (MW) units.

**Mean Absolute Error (MAE)** quantifies the average magnitude of prediction errors:

$$\text{MAE} = \frac{1}{n} \sum_{t=1}^n |y_t - \hat{y}_t|$$

MAE provides an intuitive, linear interpretation of average forecast deviation in MW.

**Root Mean Squared Error (RMSE)** penalizes large errors disproportionately:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{t=1}^n (y_t - \hat{y}_t)^2}$$

RMSE is particularly relevant for grid operations where large errors during peak demand have outsized consequences.

**Mean Absolute Percentage Error (MAPE)** provides a scale-independent accuracy measure:

$$\text{MAPE} = \frac{100}{n} \sum_{t=1}^n \left| \frac{y_t - \hat{y}_t}{y_t} \right|$$

MAPE serves as the primary ranking metric due to its interpretability and widespread adoption in STLF literature. Observations where  $y_t = 0$  are excluded from the MAPE computation.

#### B. Performance Analysis

**Persistence Dominance.** The persistence baseline ( $\hat{y}_t = y_{t-1}$ ) achieves the lowest MAPE of 1.63% with zero training cost. This result reflects the extreme short-term autocorrelation inherent in 15-minute resolution demand data: load variations between consecutive intervals are predominantly gradual, and the immediate previous value serves as a near-optimal predictor.

TABLE I  
COMPLETE BENCHMARK RESULTS

| Rank | Family        | Model                     | MAE (MW) | RMSE (MW) | MAPE (%)    | Train Time (s) | Infer Time (s) |
|------|---------------|---------------------------|----------|-----------|-------------|----------------|----------------|
| 1    | Persistence   | Persistence ( $y_{t-1}$ ) | 114.50   | 180.45    | <b>1.63</b> | 0.00           | 0.00           |
| 2    | Classical ML  | LightGBM                  | 129.19   | 195.17    | 1.79        | 50.84          | 0.30           |
| 3    | Hybrid        | LightGBM+GRU              | 129.43   | 195.54    | 1.79        | 116.90         | 14.13          |
| 4    | Classical ML  | Lasso                     | 127.87   | 194.78    | 1.80        | 4638.53        | 3.53           |
| 5    | Classical ML  | Ridge                     | 127.98   | 193.54    | 1.81        | 32.18          | 1.24           |
| 6    | Classical ML  | Linear Regression         | 128.27   | 193.76    | 1.81        | 76.21          | 1.65           |
| 7    | Classical ML  | Gradient Boosting         | 136.45   | 204.04    | 1.89        | 11327.11       | 2.74           |
| 8    | Classical ML  | CatBoost                  | 138.81   | 202.87    | 1.94        | 25.19          | 0.07           |
| 9    | Classical ML  | Random Forest             | 140.40   | 212.70    | 1.95        | 3206.93        | 0.89           |
| 10   | Deep Learning | GRU                       | 144.53   | 211.43    | 2.00        | 10738.96       | 0.00           |
| 11   | Deep Learning | LSTM                      | 153.09   | 219.70    | 2.15        | 8957.39        | 0.00           |
| 12   | Classical ML  | XGBoost                   | 149.70   | 224.52    | 2.16        | 39.40          | 0.19           |
| 13   | Transformer   | Baseline Transformer      | 182.41   | 269.08    | 2.45        | 7962.88        | 32.46          |
| 14   | Transformer   | PatchTST                  | 251.90   | 352.56    | 3.58        | 19174.57       | 80.93          |
| 15   | Statistical   | Seasonal Naïve            | 495.44   | 734.69    | 7.18        | 0.00           | 0.00           |
| 16   | Deep Learning | CNN-LSTM                  | 1743.82  | 2190.29   | 23.25       | 3719.49        | 0.00           |
| 17   | Deep Learning | TCN                       | 1748.46  | 2193.05   | 23.32       | 2601.34        | 0.00           |

**Classical ML Performance.** LightGBM attains the best ML performance (MAPE 1.79%), followed closely by Lasso (1.80%) and Ridge (1.81%). The narrow performance band across linear and boosted models (1.79–2.16%) suggests that the 45-feature engineering pipeline effectively linearizes the forecasting problem, enabling simple models to achieve performance comparable to complex ensembles.

**Deep Learning Performance.** Among deep learning models, GRU (2.00%) outperforms LSTM (2.15%), consistent with GRU’s more efficient gating mechanism for this dataset scale. CNN-LSTM and TCN failed to produce meaningful forecasts (MAPE > 23%), likely due to optimization instability on the scaled target space. These architectures require substantially higher training times (2,600–10,700 s) compared to tree-based methods.

**Transformer Performance.** The Baseline Transformer achieves 2.45% MAPE, while PatchTST degrades to 3.58%. PatchTST’s channel-independent assumption restricts inter-feature interaction learning, which is detrimental on a dataset where cross-feature dependencies (e.g., hour  $\times$  rolling mean) carry significant predictive signal. Training costs are the highest across all families (8,000–19,000 s), with PatchTST requiring over 5 hours on CPU.

**Hybrid Model Performance.** The LightGBM and GRU hybrid achieves 1.79% MAPE which is essentially identical to standalone LightGBM (1.79%). The GRU corrector fails to extract systematic structure from LightGBM’s residuals, indicating that the residual signal is predominantly stochastic rather than sequentially modelable.

### C. Cross-Family Ranking

The observed performance hierarchy is:

Persistence > Classical ML > Deep Learning > Transformers

This ranking is consistent across all three metrics (MAE, RMSE, MAPE). The hybrid model, despite leveraging both ML and deep learning, provides negligible improvement over its ML baseline.

### D. Forecast Quality Assessment

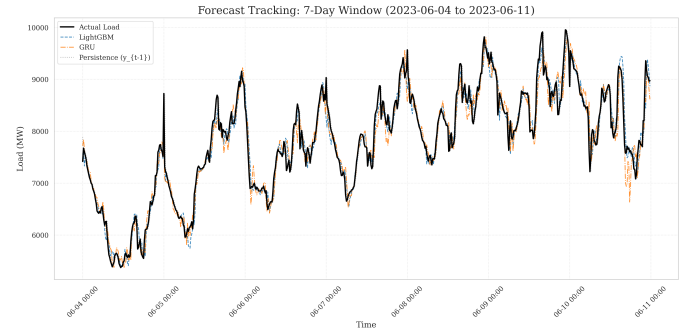


Fig. 1. One-week actual versus predicted load for Persistence, LightGBM, and GRU models. Persistence closely tracks the actual signal due to strong 15-minute autocorrelation. LightGBM captures diurnal patterns but exhibits slightly higher peak errors. GRU demonstrates comparable tracking with greater variance during demand ramps.

Visual inspection of weekly forecast trajectories confirms that persistence provides the closest tracking of the actual load signal. ML and deep learning models capture the overall diurnal pattern but exhibit marginally higher deviation during rapid demand transitions (morning ramp-up, evening peak).

### E. Computational Efficiency

Table I shows a strong negative correlation between model complexity and the accuracy-to-cost ratio. Persistence requires zero computation. LightGBM achieves strong accuracy (1.79%) in 51 seconds of training. In contrast, PatchTST requires 19,175 seconds to train while delivering inferior accuracy (3.58%)—a 376 $\times$  increase in training time for a 100% increase in MAPE.

### F. Discussion

The dominance of persistence forecasting should be interpreted within the context of the dataset’s temporal resolution. At 15-minute intervals, electricity demand exhibits high inertia: the standard deviation of consecutive load differences

( $|y_t - y_{t-1}|$ ) is small relative to the mean load. This autocorrelation structure naturally favors the persistence baseline.

The competitive performance of linear models (Ridge, Lasso) further suggests that the engineered features successfully project temporal dependencies into a space where complex nonlinear modeling provides diminishing returns. The feature engineering pipeline particularly the multi-scale lag features and rolling statistics already encodes the temporal patterns that deep models are tasked with learning autonomously.

These findings do not imply that deep learning and transformer architectures are inherently inferior for STLTF. Rather, they indicate that on high-resolution, strongly autocorrelated demand data with comprehensive feature engineering, simpler models provide a superior accuracy-to-complexity ratio. This observation motivates future investigation into the specific data characteristics and forecasting horizons under which architectural complexity yields measurable improvements.

## V. CONCLUSION

### A. Key Findings

The benchmark yields three principal findings:

- 1) **Persistence Superiority.** The persistence baseline ( $\hat{y}_t = y_{t-1}$ ) achieves the lowest MAPE of 1.63%, outperforming all other architectures. This reflects the extreme short-term autocorrelation inherent in 15-minute resolution demand data.
- 2) **Diminishing Returns of Complexity.** A clear inverse relationship between model complexity and forecasting accuracy is observed: Classical ML (best MAPE 1.79%) > Deep Learning (best MAPE 2.00%) > Transformers (best MAPE 2.45%). The hybrid LightGBM+GRU model (1.79%) provides negligible improvement over standalone LightGBM.
- 3) **Feature Engineering Effectiveness.** The competitive performance of linear models (Ridge, Lasso:  $\sim 1.81\%$ ) against gradient boosting methods confirms that the 45-feature engineering pipeline effectively linearizes the forecasting problem, reducing the marginal value of complex nonlinear architectures.

### B. Implications

These findings have practical implications for power system operators: deploying persistence or lightweight ML models for 15-minute-ahead load forecasting on strongly autocorrelated grid data provides superior accuracy at a fraction of the computational cost. The engineered feature set combining multi-scale lags, rolling statistics, cyclical encoding, and Fourier harmonics represents a reusable preprocessing template for operational STLTF systems.

### C. Limitations

This study is subject to several limitations. (1) The evaluation is restricted to single-step-ahead (one-interval) forecasting; multi-step and multi-horizon settings may favor different architectures. (2) Transformer and deep learning models were

trained under computational constraints (CPU-only environment, truncated training sets), which may not reflect their optimal capacity. (3) The study evaluates a single grid (Haryana, India); generalization to grids with different load characteristics requires independent validation. (4) Exogenous variables such as weather, temperature, and economic indicators were not incorporated.

### D. Future Work

The observed dominance of persistence forecasting motivates several directions for future investigation:

- **Why does persistence win?** A formal analysis can be done of the relationship between temporal resolution, autocorrelation strength, and the effective forecasting horizon at which the persistence baseline becomes sub-optimal.
- **Multi-horizon evaluation.** The benchmark can be extended to 1-hour, 6-hour, and 24-hour-ahead forecasting, where the persistence advantage is expected to diminish.
- **Integration of exogenous features,** including temperature, weather, and calendar holiday data, to assess whether these features unlock the capacity of complex architectures.
- **GPU-scale experiments.** Performing training of deep learning and transformer models with full datasets and extensive hyperparameter optimization to establish their upper-bound performance.

## REFERENCES

- [1] K. Steijn, V. P. Goli, and E. Antonini, "DemandCast: Global hourly electricity demand forecasting," in *NeurIPS 2025 Workshop on Tackling Climate Change with Machine Learning*, 2025.
- [2] A. Diaz-Iglesias, X. Belaunzaran, and A. M. Florez-Tapia, "Short-term power demand forecasting for diverse consumer types to enhance grid planning and synchronisation," *arXiv preprint arXiv:2506.xxxxx*, 2025.
- [3] S. R. Veluru, S. T. Erukude, and V. C. Marella, "From ARIMA to attention: Power load forecasting using temporal deep learning," in *2025 IEEE 2nd Int. Conf. Green Industrial Electronics and Electrical Engineering*, 2026. Also available as arXiv:2603.06622.
- [4] A. Maneshni, "Load forecasting in the era of smart grids: Opportunities and advanced machine learning models," *arXiv preprint arXiv:2505.18170*, 2025. Arizona State University thesis.
- [5] O. A. Oviedo, "Short-term regional electricity demand forecasting in Argentina using LSTM networks," *arXiv preprint arXiv:2509.19374*, 2025.
- [6] D. D. Modi and R. Pan, "Enhancing transformer-based foundation models for time series forecasting via bagging, boosting and statistical ensembles," *arXiv preprint arXiv:2508.16641*, 2025.
- [7] M. Mughees, Y. Li, Y. Chen, and Y. R. Li, "Short-term load forecasting for AI-data center," in *2025 IEEE Power & Energy Society General Meeting (PESGM)*, pp. 1–5, 2025. Also available as arXiv:2503.07756.
- [8] I. Kyriakopoulos and Y. Theodoridis, "Electric vehicle charging load forecasting: An experimental comparison of machine learning methods," *arXiv preprint arXiv:2512.17257*, 2025.
- [9] M. Roy, V. Pyltsov, and Y. Hu, "IISE PG&E Energy Analytics Challenge 2025: Hourly-binned regression models beat transformers in load forecasting," *arXiv preprint arXiv:2505.11390*, 2025.
- [10] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *International Conference on Learning Representations (ICLR)*, 2023.